

Audit before you architect

Pick a real candidate problem from your team. Run it through the rubric, then pressure-test it against the six failure patterns. ChatGPT is your design partner; you are the architect.

WHAT YOU WILL PRODUCE

A one-paragraph problem statement, a quadrant classification with justification, and a six-row failure-pattern table showing where each pattern would bite your candidate problem.

Format: Write the final output in a single ChatGPT message you can copy out at the end. The instructor will sanity-check it on return.

1 Pick your candidate problem (3 min)

Pick one real problem from your current work that someone has called "an AI use case." Examples that count:

- "We want AI to handle Tier-1 support tickets."
- "Automate vendor invoice processing."
- "Build an agent to plan sales territory routes."
- "Use AI to triage customer complaints."

Write the problem in **one sentence**. If it takes three sentences, it has scope creep — narrow it.

2 Open ChatGPT and run the classifier (8 min)

Open a fresh ChatGPT session. Paste this prompt verbatim, replacing the placeholder with your one-sentence problem from Step 1.

PROMPT TO PASTE INTO CHATGPT

You are helping me apply the four-quadrant rubric for AI systems. The two axes are:

- Axis 1: Is the next step decided by code (deterministic) or by the model (non-deterministic)?
- Axis 2: Does the system need a single step or multiple steps?

The four quadrants are:

- Deterministic + Single step = AUTOMATION (cron, Zapier, webhook)
- Deterministic + Multi-step = WORKFLOW (Airflow, Step Functions)
- Non-deterministic + Single = LLM CALL (copilot, classifier)
- Non-deterministic + Multi = AGENT (only this is an agent)

My candidate problem is: [PASTE YOUR ONE-SENTENCE PROBLEM HERE]

Do exactly three things:

1. Ask me up to three clarifying questions if anything is ambiguous. WAIT for my answers before continuing.
2. Classify the problem into one of the four quadrants. Justify in two sentences.
3. If the classification is anything OTHER than AGENT, recommend the simpler shape and stop there. If it IS an agent, list the multi-step decisions the system would make.

WHAT GOOD LOOKS LIKE

ChatGPT asks clarifying questions before classifying. It does not default to "agent" — most candidates demote to workflow or LLM call. If you got demoted, that is the win.

3 Pressure-test against the six failure patterns (12 min)

Whatever shape ChatGPT classified your problem as, paste the next prompt to map the six failure patterns onto it.

PROMPT TO PASTE NEXT

Now I want to pre-mortem this design. Here are the six enterprise failure patterns from my training:

1. LOOPING – same tool called repeatedly with no terminal state
2. SILENT TOOL FAILURE – tool errored, agent reports success
3. HALLUCINATED STEP – agent claims an action it never executed
4. CONTEXT POISONING – memory leaks across users or sessions
5. STATE DRIFT – multi-step workflow loses track of progress
6. UNSAFE AUTONOMY – irreversible action without an approval gate

For my specific problem, produce a six-row table with these columns:

- Pattern
- Where it would bite my system (be specific to my domain)
- One concrete defence (technical, not generic advice)

Rules:

- Be domain-specific, not generic. Name the actual tool, user, or data flow.
- If a pattern genuinely does not apply (e.g., looping for a single-step system), say so explicitly – do not pad.
- Each defence must be implementable, not aspirational.

4 Critique the output and harden one row (5 min)

Pick the **most likely** failure pattern for your system. Send this follow-up to harden it:

PROMPT TO PASTE NEXT

From the six patterns, which ONE is most likely to surface in production for my specific problem? For that pattern only:

- Give me a concrete signal I could put into observability (a log line, metric, or alarm condition) that would catch it.
- Give me a single QA test I could write today that would fail if this defence regressed.

Be ruthlessly specific. No "add monitoring" – tell me what to monitor.

5 Verify and capture (2 min)

Before the session resumes, run this self-check. Tick each box in your notebook.

- ☐ My problem statement is exactly one sentence long.
- ☐ ChatGPT asked clarifying questions before classifying (if it did not, push back and ask why).
- ☐ The classification names a specific quadrant with a two-sentence justification.
- ☐ The failure-pattern table has six rows. At least one pattern is marked "does not apply" with a reason.
- ☐ Every defence in the table is implementable, not aspirational.
- ☐ My observability signal is a concrete log line, metric, or alarm — not "add monitoring."
- ☐ I can name the most likely failure pattern for my system and defend the choice.

WHEN THE INSTRUCTOR RETURNS

Be ready to answer in 30 seconds: **What is your problem, what quadrant did it land in, and which failure pattern is the biggest risk?** If you got demoted from agent to workflow, that is the most interesting answer in the room.